hdoc day 47 by trishil chauhan

C Programming - Ceil Index in a Sorted Array

Aim: Read a sorted variable-length integer array and a target, then find the first index whose element is greater than or equal to the target. Pass the array by pointer to **ceil_index()**.

Complete C Program

```
#include <stdio.h>

/* Returns the first index with arr[index] >= target. */
int ceil_index(const int *arr, int n, int target) {
    int left = 0, right = n - 1;
    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (*(arr + mid) >= target) {
            answer = mid;
            right = mid - 1;  /* Look for an earlier index. */
        } else {
            left = mid + 1;
        }
    }
    return answer;
}

int main(void) {
    int n, target;

    printf("Enter array size: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }

    int arr[n];  /* Variable-length array (C99). */
    printf("Enter %d elements in ascending order:\n", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%d", arr + i) != 1) {
            printf("Invalid input.\n");
            return 1;
        }
        if (i > 0 && *(arr + i) < *(arr + i - 1)) {
            printf("Error: array must be sorted ascending.\n");
            return 1;
        }
    }

    printf("Enter target: ");
    if (scanf("%d", &target) != 1) {
        printf("Invalid target.\n");
        return 1;
    }

    int index = ceil_index(arr, n, target);
    if (index == -1) {
        printf("-1\n");
    } else {
```

```
        printf("Ceil element: %d\n", *(arr + index));
        printf("Ceil index: %d\n", index);
    }
    return 0;
}
```

Sample Output 1 - Ceil with duplicates

```
Enter array size: 7
Enter 7 elements in ascending order:
1 3 5 5 5 8 10
Enter target: 5
Ceil element: 5
Ceil index: 2
```

Sample Output 2 - No ceil exists

```
Enter array size: 4
Enter 4 elements in ascending order:
2 4 6 8
Enter target: 9
-1
```

Explanation

The binary search records a candidate whenever arr[mid] is at least the target and continues searching to the left. This guarantees the first occurrence, even with duplicates. If no candidate exists, the function returns -1. Indices are zero-based. Time complexity: O(log n); auxiliary space: O(1).